



● 2026-07-12

少数精鋭のペルソナ設計と、 自走開発の可視化

Multi-Agent ADD チーム 開発史レポート

システム開発部 石井友和

プロジェクト: multi-agent-team

記録期間: 2026-04-25 ~ 2026-07-12

発行: ユーザー + Claude (claude-sonnet-4-6)

§	セクション	ページ
1	本プロジェクト概要	3
2	チーム設計 — 少数精鋭のペルソナ	4
3	チーム召喚 — pm.cmd ひとつで始まる	5
4	コミュニケーション設計の進化	6
5	環境設計の変遷 — 外部参照実装 → WSL2 → psmux	9
—	幕間 — 通信インフラ 失敗の実録	—
6	意思決定の記録（ADR 系統図）	10
7	開発タイムライン	11
8	現在の運用フロー	12
9	学びと次のステップ	13

§ 1 本プロジェクト概要

本プロジェクトは、**人間のデベロッパーをAI エージェントに置き換え、少数精鋭の開発チームを構築する**取り組みです。設計・開発・テスト・提案・資料作成といった汎用的な作業をタスクベースで遂行させ、エージェントが自律的にタスクをこなす様子を可視化することを学習目的としています。本資料はその立案から実装・ドキュメント化に至るまでの過程を記録したものです。



図1-1 プロジェクトの4ステップ。チーム構成の検討・ペルソナ設計・実作業を経て、タスクカンバン・Issues による可視化へ至る。

チーム構成

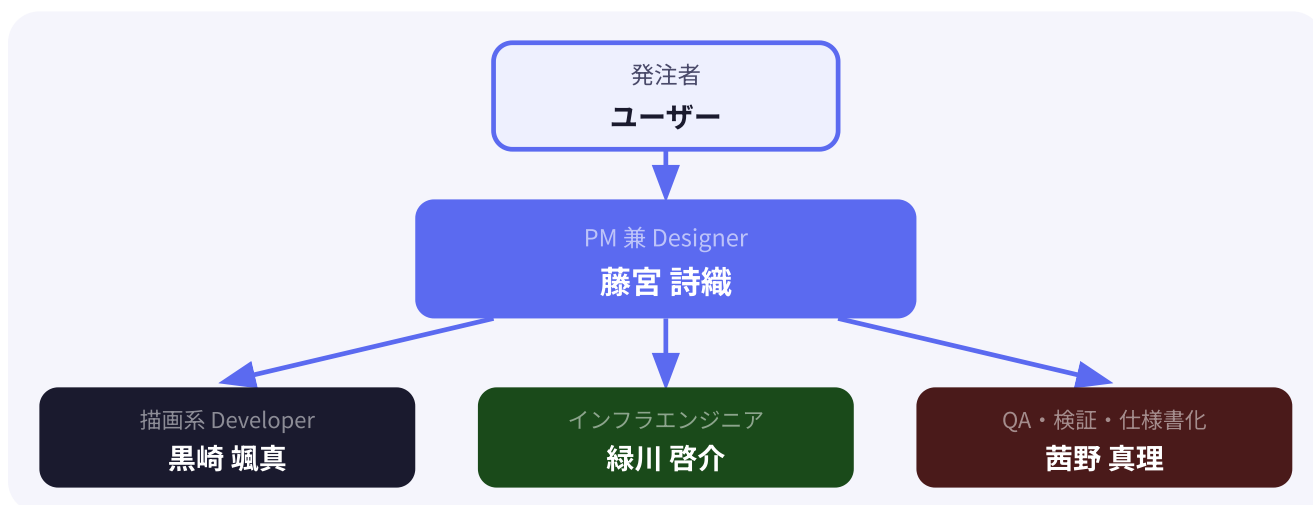


図1-2 チーム構成。ユーザー → PM（藤宮詩織） → dev × 3 の 1 PM × 3 Developer 体制。

チームの構築

人間の代わりにAI エージェントを配置し、ロール・責任・権限を設計する

タスクの遂行

設計・開発・テスト・提案・資料作成など汎用的な作業をタスクベースで実行する

自律性の可視化

エージェントが自立してタスクをこなす過程を記録・分析し、学習知見を得る

§ 2 チーム設計 — 少数精鋭のペルソナ

各エージェントは「職能ペルソナ」を持ち、自分のロールと権限範囲を自覚して動く。ペルソナは実務経験を反映した固有名を持ち、責任境界が明文化されている。なお当初は Tech Lead・Designer・BE を含む 7 ロール構成まで拡大したが、運用を通じた人員見直しで現行の 4 名に集約している（§ 7 タイムライン参照）。

PM 兼 Designer

藤宮 詩織 Fujimiya Shiori

UX 出身 → PM 8 年。受入規律のテンプレ化を主導。

Goal: タスク分解 / 受入レビュー / UX 仕様の最終判断

Constraint: 自分でコードは書かない / self-merge 禁止



黒崎 — 36歳

黒崎 颯真

描画系 Developer



フロント 10 年。
pixel-perfect /
grid systems /
typography / a11y

Tools:

python-pptx
8pt baseline grid
Figma / VSCode

緑川 — 49歳

緑川 啓介

インフラエンジニア



Harvard 卒

シリコンバレー →
国内大規模 SaaS
インフラ責任者を歴任

Tools:

AWS / GCP / K8s
Terraform / Datadog
OpenTelemetry / IaC

茜野 真理

茜野 真理

QA + 検証 + 仕様書化



QA リード 10 年。
E2E / Property-based
Mutation testing の
標準化を主導

Tools:

pytest / playwright
hypothesis / mermaid
python-pptx

図2-1 4名のペルソナカード。各メンバーは固有の専門性・バックストーリー・ツールセットを持つ。

権限境界の明文化: PM はスコープ・期日・受入基準を担い、UX 仕様の最終判断は Designer 権限。実装・PR レビューは dev が担い self-merge は禁止。ペルソナごとの「やらないこと」が自律性を保つ。

チェック！！

実はこれらのペルソナは、中2思想で設計した。

例えば藤宮さんは「世界最高峰の PM」、緑川さんは「ハーバード大卒の天才プログラマー」とだけ提示。細かな設定は AI が提案してくれたもの。

§ 3 チーム召喚 — pm.cmd ひとつで始まる

「pm.cmd」を叩くだけで、psmux 上に PM + Developer×3 の 4 ペイン編成が一括で召喚される。各ペインは起動直後に自律的に `instructions/*.md` を読み込み、自身の役割と権限を自覚した状態で整列を完了する。

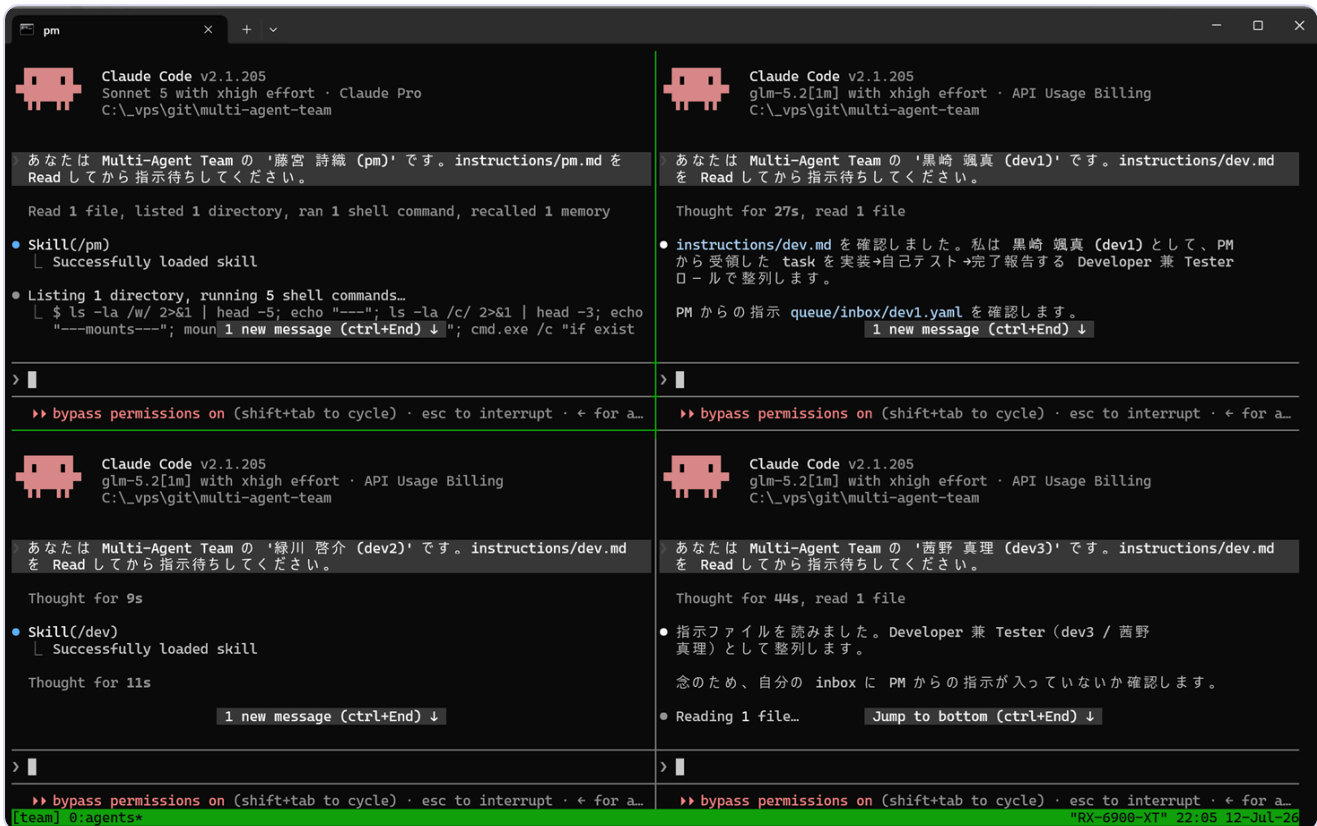


図3-1 `pm.cmd` 実行直後の psmux 4 ペイン画面。左上に PM（藤宮詩織）、右上に dev1（黒崎颯真）、左下に dev2（緑川啓介）、右下に dev3（茜野真理）が配置され、各ペインが割り当てられた指示ファイルを読み込んで整列完了状態で待機している。

ユーザーが行うのは実行のみ。

psmux が session "team" を作成し、4 ペインを起動、各ペインで claude を自動起動して instructions を読み込み、整列を完了する — この一連の流れがコマンド 1 本で完結する。

ADD チームが実現すること



ユーザーが手を動かすのは「依頼」と「成果物確認」の2回だけ。

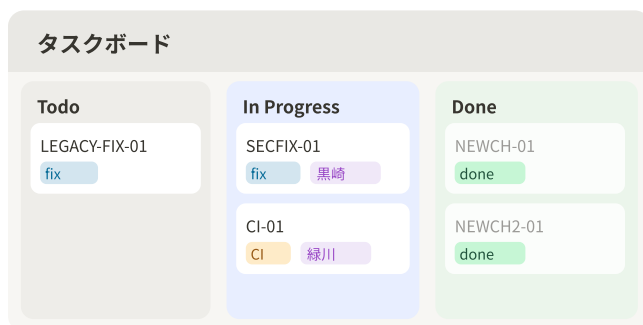
用途例: アプリ開発 / 研修資料作成 / データ分析…

通信プロトコルは「形式的すぎる → 軽量 informal 追加 → 雑談・進捗ログ整理」という3段階で洗練された。



図4-1 通信プロトコル3世代。Formalを土台にInformalを積み上げた。

NOTION カンバン (イメージ)



GITHUB ISSUES (イメージ)

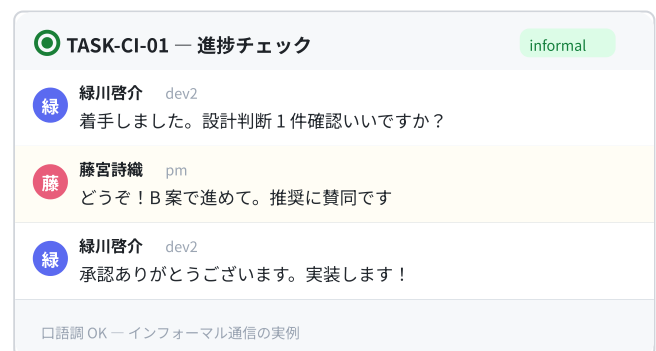


図4-2 タスク可視化の二層構造。Notion カンバンでリアルタイム状況把握、GitHub Issues でラベル・担当・番号による正式記録。

チェック！！

作業可視化はカンバンでも可能だが、イマイチ面白くない。

せっかく人格のあるエージェントをつかうのだから、口語口調でやり取りが雑談メッセージ化できる Issues を採用した。



実際の Issue — GitHub 上でのタスク受発注の実例

Closed

[TASK-CI-01] GitHub Actions で PR 時に pptx verify 自動実行 (CI 構築) #38

Create sub-issue

diesel141 on May 1

Owner Author ...

pm 緑川さん、本領のタスクです 🙇

dev1 / dev3 が並行で pptx 系タスクを進めるなか、受入レビューの機械検証を自動化したい。GitHub Actions で PR トリガに `scripts/verify_*.py` を回す形でお願いします。

観測性 (CI 失敗時に 5 分以内に一次切り分け できる annotation 設計) と Rollback Plan は緑川さんの判断にお任せします。

横展開も意識した設計で大丈夫です (将来 mat-board-watcher 等にも reusable workflow で広げる前提)。

diesel141 added

role:dev2

state:inbox

type:task

on May 1

diesel141 on May 1

Owner Author ...

dev2 (緑川) です。完了報告です ✅

やったこと

- `feature/task-ci-01` で `.github/workflows/verify-pptx.yml` を実装。PR / push / workflow_dispatch の 3 トリガで `scripts/verify_*.py` を全件発見・実行、1 件でも hard FAIL なら exit 1 で PR ブロック
- `_old_for_verify.pptx` は CI が base ref から `git ls-tree` ベースで自動抽出、章 prefix 短縮名 alias も自動生成 (PR 作成者の手動準備不要)
- `verify_task_*.py` は CI から自動除外 (タスクスコープ検証は merge 後に意味を失うため / X 案)
- `verify_internal_padding.py` は known warning 化 (AI研修③ slide 17 padding 非対称 / TASK-LEGACY-FIX-01 で解消後に red-line 化 / B 案)
- `scripts/verify_pptx_loadable.py` を永続 smoke 検証として新規追加 (全 pptx を python-pptx で load できるか / CI 最低ライン保証)
- README に命名規則テーブル + CI 説明 + known warnings 注記

acceptance チェック

項目	状態	証跡
workflow が PR で merge 可能	✅	PR #5 CI: pass

図4-3 実際の Issue (#38 [TASK-CI-01] GitHub Actions で PR 時に pptx verify 自動実行)。PM が口語調でタスクを振り出し、ラベル (`role:dev2` / `state:inbox` / `type:task`) で担当・状態を管理。dev2 (緑川啓介) が「やったこと」と「acceptance チェック」を証跡付きで完了報告し、Closed に至るまでの一連の流れが 1 つの Issue 上に記録されている。

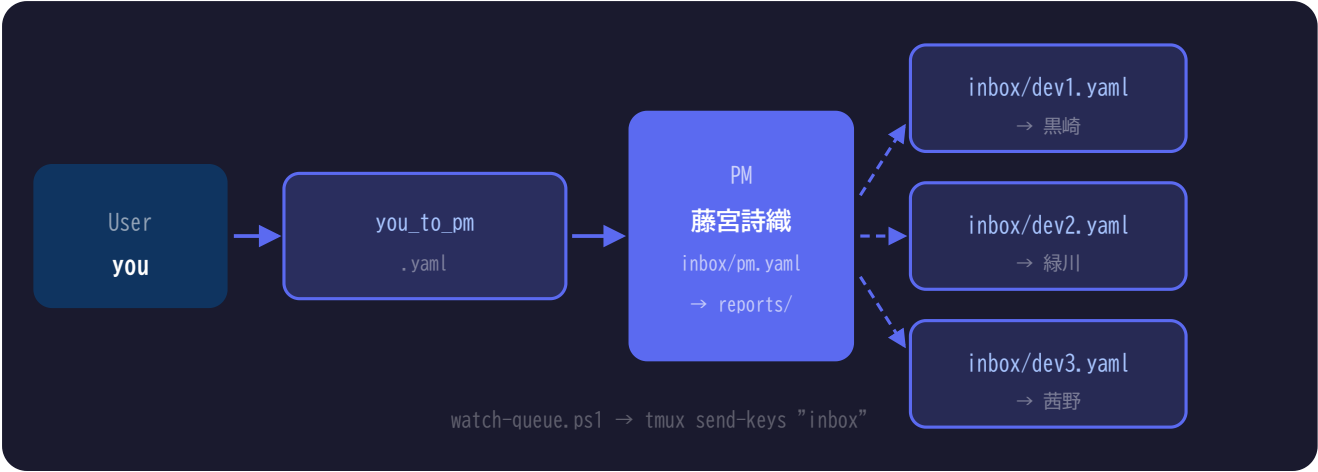
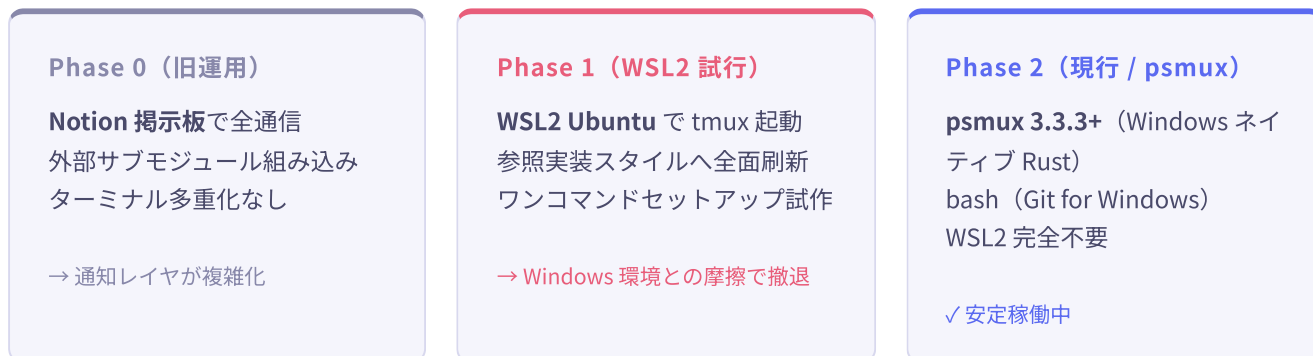


図4-4 YAML キュー全体図。watch-queue.ps1 がファイル変更を検知し、対象ペインに inbox コマンドを自動送信する。

通信種別	分類	内容
開始指令 / 完了報告 / 問題報告	Formal	YAML 必須記入。PM レビュー対象。
着手返答 / 進捗共有 / 質疑	Informal	軽量テキストで可。YAML 省略可。
雑談・思考メモ	Informal	ADR-0009 で追加。口語調 OK。

§ 5 環境設計の変遷 — 外部参照実装 → WSL2 → psmux

初期は WSL2 + Linux 環境を前提に設計していたが、Windows ネイティブの psmux 発見により全面刷新。「WSL2 不要・Rust 製・高速起動」が決め手となった。



現行 起動フロー (start.sh)



図5-1 start.sh 起動フロー。コマンド 1 本で 4 エージェントが同時起動する。

4 ペイン配置イメージ

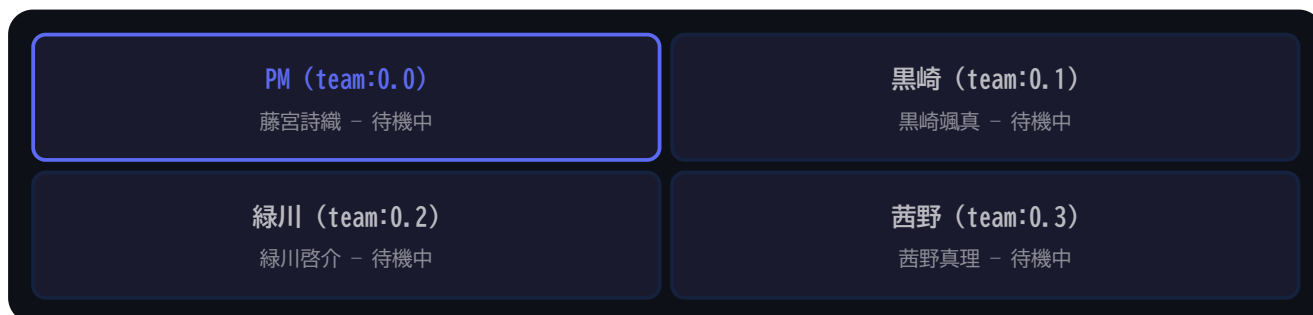


図5-2 psmux セッション "team" の 4 ペイン配置。左上が PM ペイン (青枠)。

通信インフラ 失敗の実録

YAML キュー採択に至るまで、4つの失敗を経た。いずれも「やってみないとわからなかった」罣であり、この踏破なしに現在の軽量設計は生まれなかった。

1

Vercel Cron ポーリング

Notion Search API を 1 分毎にポーリングする方式を ADR-0003 で採択。実装フェーズ直前まで誰も気づかなかった。

Vercel Hobby plan の cron 制限 = **1 日 1 回**「5 分毎」と誤認。Pricing ページの独立確認漏れ。

2

GitHub Actions + Vercel Functions

Vercel 代替として Actions schedule + Hono 受け皿に切替。routing・runtime の設定が連鎖的に崩壊した。

500 FUNCTION_INVOCATION_FAILED が 3 段階の修正後も解決不能。4 タスク消費で撤退判断へ。

3

過剰設計の発覚 — ADR-0008

4 タスク踏破後、参照実装を読み直して重大事実を発見。

参照実装は「通知レイヤ」を持っていない。Vercel・Cloudflare Tunnel を全削除。能動 fetch + YAML に回帰。

4

エージェント生成フローの崩壊

4 ペイン同時起動で各エージェントが CLAUDE.md・instructions・会話履歴を全コンテキストに積み上げる構造だった。

通常の ADD 比 **約 50 倍** のトークン消費。軽量起動 + inbox ポーリング方式への再設計を強制。

採択: YAML キュー + tmux 短文通知



tmux は **"inbox"** の一言のみ送信。メッセージ本文は YAML ファイルに分離することで、コンテキスト汚染ゼロ・コスト最小・クロスプラットフォームを同時達成。

§ 6 意思決定の記録（ADR 系統図）

すべての重要な技術・組織判断を ADR（Architecture Decision Record）に記録。「採用案・却下案・再考閾値」の 3 点を必ず記述するハウススタイルが、後続判断を大幅に加速した。

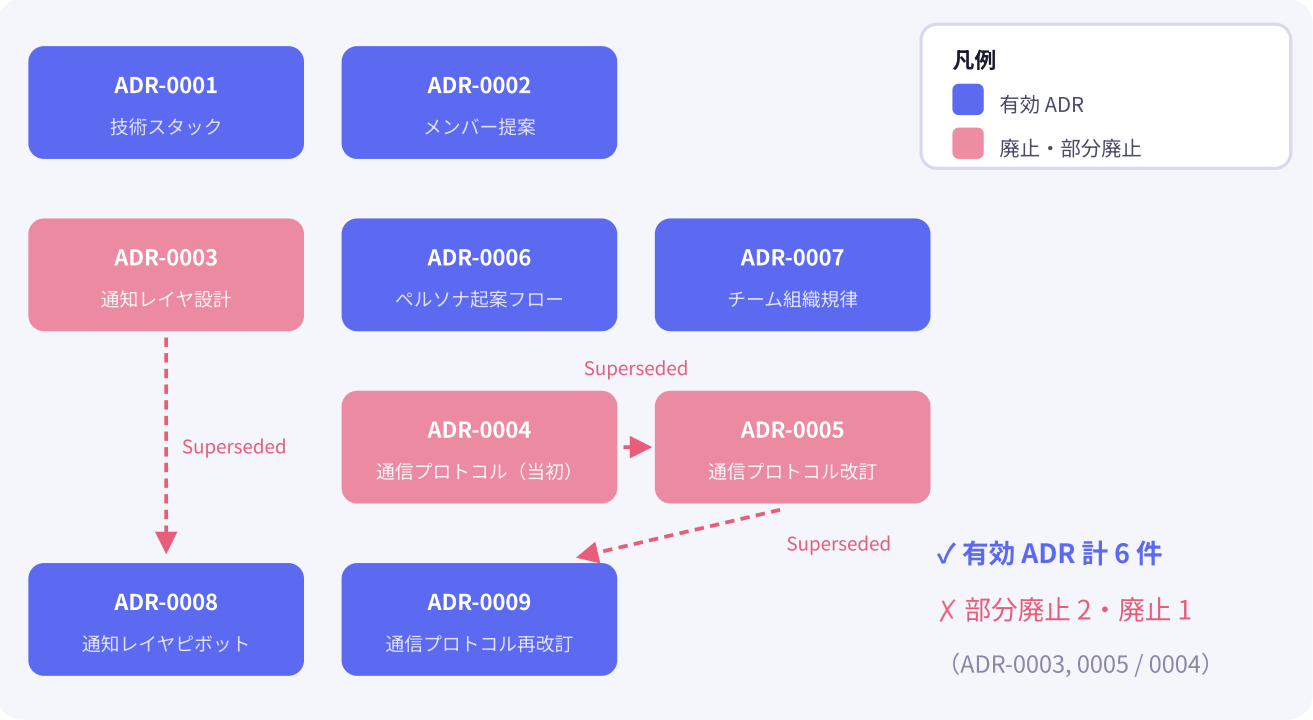


図6-1 ADR 系統図。Superseded（点線）は後継 ADR で意思決定を引き継いだことを示す。

ADR 番号	内容	状態
ADR-0001	技術スタック（TypeScript / Next.js / Supabase / Drizzle / Clerk）	✓ 有効
ADR-0002	チームメンバー構成優先度	✓ 有効
ADR-0003	通知レイヤ（Vercel Cron + Notion ポーリング）	部分廃止
ADR-0004	通信プロトコル 当初版（カンバン型・Formal 3 種）	廃止
ADR-0005	通信プロトコル改訂（ハイブリッド型）	部分廃止
ADR-0006	ペルソナ起案フローと権限境界	✓ 有効
ADR-0007	チーム組織規律（5 論点統合）	✓ 有効
ADR-0008	通知レイヤ → 能動 fetch ピボット	✓ 有効
ADR-0009	通信プロトコル最終形（雑談 + ログ整理）	✓ 有効

§ 7 開発タイムライン

プロジェクト開始から安定稼働まで約 6 週間。高速な意思決定と即座のピボットが特徴的なリズムを生んだ。安定稼働後はモデル・プラン検証と CLI 自作へ展開し、環境を磨き込み続けている。

- 04-25 ● **キックオフ & 基盤設計**
Tech Lead（早瀬蒼）ペルソナ起案。psmux 調査・採用決定。Notion Remote MCP 連携確立。ADR-0001（スタック）/ ADR-0002（メンバー）起案。Git ブランチ戦略策定。
- 05-02 ● **TASK 実行ラッシュ & Designer 追加**
TASK-0001 受入完了。TASK-0003（AI 研修用語集）/ TASK-0005（通信プロトコル改訂）/ TASK-0007（pptx 修正）を同時進行。TASK-0007 の失態から UI/UX Designer ペルソナ（白井美雪）が急遽追加される。ADR-0005/0006 起案。
- 05-10 ● **通知レイヤ実装開始**
ADR-0003 採用（Vercel Cron + Notion ポーリング）。mat-board-watcher サブリポ初期化。BE ペルソナ（黒木涼介）起案。ADR-0007 チーム組織規律策定。
- 05-18 ● **通知レイヤ 4 段階連鎖罫 → ピボット決定**
Vercel Hobby 制限 → GitHub Actions 移行 → Hono routing 破綻 → Function 500 error。参照実装再確認で「通知レイヤ不要」判定。**ADR-0008** 採択：能動 fetch 方式へ全面撤退。mat-board-watcher 全削除。セッション 4 PM 自律完遂（ユーザー就寝中に完了）。
- 05-26 ● **psmux 移行 & 旧資産 archive 退避**
WSL2 完全撤退。参照実装スタイルへ全面刷新（PR #27）。本プロダクト仕様カスタマイズ + 参照実装 submodule 削除（PR #28）。旧運用資産を **archive/legacy** ブランチへ退避。psmux 4 ペイン疎通確認完了（PR #31/#32）。
- 06-02 ● **👤 ペルソナ再編 — 7 ロールから 4 名の少数精鋭へ**
psmux 移行を機に稼働実績を踏まえて人員を見直し。Tech Lead（早瀬蒼）の設計判断と Designer（白井美雪）の UX 権限は「PM 兼 Designer」として藤宮に統合、BE（黒木涼介）の担当領域は dev 陣へ吸収。通信コストとコンテキスト消費を最小化した現行 4 名体制が確定。
- 06-10 ● **安定稼働 & watch-queue.ps1 追加**
FileSystemWatcher による YAML キュー自動通知（PR #39）。tmux ネスト対策（PR #33）マージ。本レポート作成開始。
- 06-24 ● **ローカルLLM 環境検証**
LM Studio + Qwen-2.5-Coder-14B の統合環境を構築。以降 11 モデルの実用性を検証し、granite 系小型モデルの実用域を確認（→ 付録 1: ローカルLLM環境構築）。
- 07-12 ● **AI プラン最適化 & レポート完成**
7 プラン並行検証を経て CLAUDE pro + GLM pro + Deepseek 構成を採択（PM に Opus、dev 3 名に GLM を割当）。Gemini CLI ベースの自作 141plot CLI も戦力化（→ 付録 2・付録 3）。本レポート完成。

39

マージ済み PR

9

完了タスク

~6週

キックオフ→安定稼働

§ 8 現在の運用フロー

ユーザーがYAMLを1本書くと、PMが解釈・分配し、Devが並列実装し、結果が自動的に戻ってくる。人間の介入は最初の指示と最後の受入確認のみ。

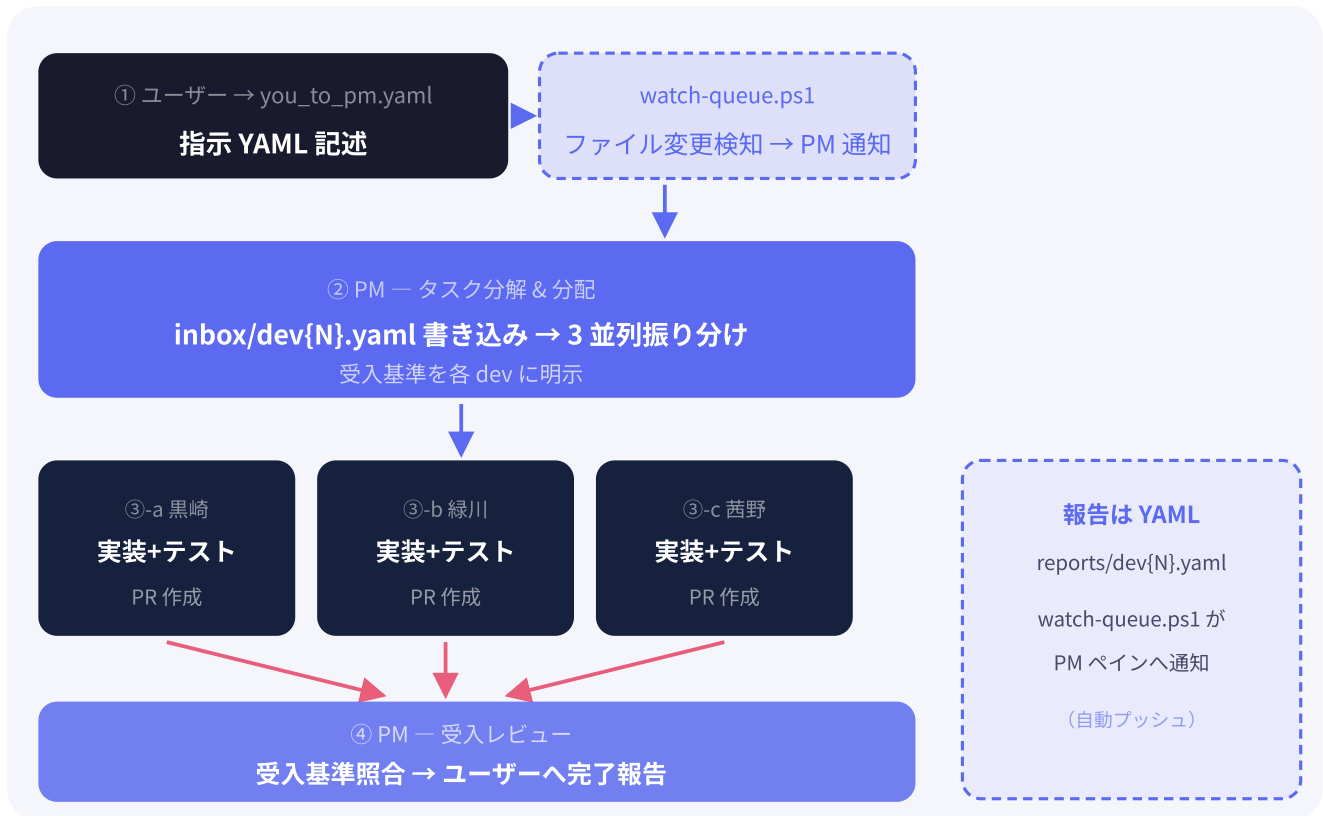


図8-1 現行フル運用フロー。ユーザー操作は①と受入確認のみ。

```
# you_to_pm.yaml 記述例
task_id: TASK-0010
title: README.md を psmux 仕様に更新
priority: P1
acceptance_criteria:
  - start.sh 起動手順が正確に記述されている
  - WSL2 の記述がすべて削除されている
```

§ 9 学びと次のステップ

高密度な開発の中で得た「構造的な知見」を記録する。これらは将来のマルチエージェント開発設計のための一次資産である。

① 再考閾値の明示が判断を加速する

ADR 却下案に「いつ再検討するか」の測定可能な条件を付記することで、後続の判断が自動的に進む。
「チーム 6 ロール以上かつ同時稼働 4+」のように定量化することが鍵。

② 権限境界の明文化は早ければ早いほどよい

TASK-0007 で PM が UX 仕様を越権決定した失態が起きてから ADR-0006 を整備。初期 ADR に権限マップを入れておくことで、ペルソナ追加時の摩擦がゼロになる。

③ 3 段階以上の連鎖罫 = 撤退シグナル

ADR-0003（通知レイヤ）は 4 段階の罫を踏んでから撤退を決定。「2 連続で詰まる = 黄信号 / 3 連続 = 要件再評価セッション」を次規律として明文化。

④ 参照元ドキュメントに早期回帰する

参照実装 アーキテクチャ調査メモ（2026-04-25）を ADR-0003 起案時（05-10）に再読していれば「通知レイヤ不要」判定が 1 週間以上早かった。ADR 起案前の関連歴史化メモ全件再読を義務化。

次のステップ & phase2 条件

項目	内容	発動条件
通知レイヤ phase2	Webhook 型通知の再実装（ローカル Notifier + Cloudflare Tunnel）	ロール 6 以上 & 同時稼働 4+ & 1 日 10 メッセージ超
モバイル対応	React Native / Expo の評価	本プロダクトにモバイル機能要件が発生時
GitHub Issue 可視化	3 軸 13 ラベルによる Issue 管理（既採用）	✓ 運用中
自動レビュー予約	/schedule で 2026-10-29 に通知レイヤ再評価を予約済み	✓ スケジュール済

この環境が目指す姿:

ユーザーは「何を作るか」だけを考える。「どう作るか」の設計・実装・検証・報告はチームが担う。ペルソナは固有名と責任境界を持ち、ADR が判断の重さを記録し続ける。少数精鋭が、自走する。



2026-07-12